

07 - Frontend Architecture for the First PWA

Local-first light peer frontend, Framework7 shell, Dexie event/outbox store, PGLite local query/search engine, and hybrid-ready boundaries

Document status and decision summary

This document adds the first PWA frontend architecture to the local-first P2P architecture set. It is an implementation plan for the first user-facing app surface. It does not replace the full-peer architecture. It defines the PWA as a local-first light peer that can later interoperate with native/Bare/Holepunch full peers and persistent availability peers.

Decision: the first app can be a PWA, but the PWA must be local-first. It writes locally first, renders from local materialized views, queues mutations locally, verifies signed data locally, decrypts authorized payloads locally, and syncs opportunistically through bridge/super-peer infrastructure. It is a light peer, not a full infrastructure peer.

- Frontend shell: Framework7 React, TypeScript, Vite.
- Local-first app storage: Dexie for signed event storage, mutation outbox, cached materialized views, identity state, contacts, petnames, pending media, and sync offsets.
- Committed local query/search engine: PGLite for relational projections, local full-text search, hybrid-search preparation, and eventual vector/semantic search modules where appropriate.
- Remote/bridge reads: TanStack Query only for bridge/server/public-index state, never as the local-first source of truth.
- Offline shell: Workbox/vite-plugin-pwa or Serwist, with explicit mutation retry logic because iOS Safari does not support standard Background Sync.
- Native path: Capacitor-ready monorepo boundaries from day one, but Capacitor/native packaging is not required for v1.
- Animation: Framework7 built-ins first, CSS transitions second, Motion for custom polish, @use-gesture only for genuinely advanced gestures.

Correct mental model

The PWA should not be a thin online client. It should be a local-first protocol client. The distinction is between local-first capability and full P2P infrastructure capability.

Role	Meaning	Expected in v1 PWA?
Local-first light peer	Owens local app state, writes locally first, verifies data locally, queues mutations, renders from local views, syncs later.	Yes. This is mandatory.
Full peer	Runs the complete P2P engine, long-lived replication, native Holepunch stack, media seeding, DHT participation where available.	No. This is future native/Bare/full-peer work.
Super peer / availability peer	Persistent infrastructure peer for bridge access, public indexing, seeding, encrypted availability, and Delivery Service-like coordination.	No. PWA consumes it; it does not replace it.

Rule: the PWA local database implementation is an adapter over protocol objects. It must not become a browser-only protocol that future full peers have to imitate. The canonical model remains signed events, identity control events, namespace records, media manifests, bridge streams, and future full-peer logs.

Recommended frontend stack

Layer	Chosen/default technology	Purpose	Important boundary
App shell	Framework7 React + TypeScript + Vite	Mobile app grammar: pages, views, tabs, sheets, modals, swipe-back, iOS/Material themes, touch-first layout.	Framework7 owns UI shell semantics, not protocol or sync truth.
Design system	Apple HIG-inspired tokens + CSS variables	Safe areas, nav/tab heights, materials, typography, spacing, radius, hairlines, motion tokens, dark/light themes.	Do not scatter Apple-like constants across screens.
Local event/outbox store	Dexie	Signed events, mutation outbox, local identity state, contacts/petnames, cached views, sync offsets, pending media metadata.	Dexie is local-first storage, but protocol objects remain canonical.
Local relational/search engine	PGLite	Relational projections, full-text	PGLite indexes/projections are

Layer	Chosen/default technology	Purpose	Important boundary
		search, local query surfaces, hybrid-search foundation, future pgvector/semantic modules.	derived and rebuildable unless explicitly promoted later.
Bridge/server-state reads	TanStack Query	Fetch/cache bridge stream metadata, public profiles, namespace lookups, public indexes, media manifest fetches.	Do not use it as the local-first state model.
Service worker/offline shell	vite-plugin-pwa + Workbox or Servist	App shell precache, offline fallback, route chunks, assets, safe static data, update prompts.	No blind caching of private decrypted data or sensitive authenticated responses.
Animation/gestures	Framework7 built-ins + Motion + optional @use-gesture	Native-like transitions, custom microinteractions, advanced gestures only when needed.	Avoid gesture soup and excessive glass/motion.
Native readiness	Capacitor-ready app boundaries	Future native haptics, biometrics, share targets, notifications, file/media handling.	Do not require native packaging for v1.

PWA package architecture

```

/apps/pwa
  Framework7 React app, Vite, service worker, PWA manifest

/apps/capacitor-ios
/apps/capacitor-android
  Empty or minimal initially; kept for future native packaging

/packages/design-tokens
  Apple-inspired tokens, platform themes, safe-area constants

/packages/ui
  Shared components wrapping Framework7 where useful

/packages/platform
  Runtime/capability detection: iOS, Android, desktop, browser, standalone PWA, Capacitor

/packages/protocol
  Signed event schemas, identity events, namespace records, media manifests, source refs

/packages/crypto
  Key generation, signing, verification, encryption/decryption, key-storage adapters

/packages/local-store
  Dexie schema, event store, outbox, materialized views, sync offsets

/packages/local-db-pglite
  PGLite schema, relational projections, local FTS/search, future pgvector/semantic indexes

/packages/local-apply
  Deterministic validation and projection pipeline for signed events

/packages/sync-client
  Bridge transport, Durable Stream reader, WebSocket transport, WebRTC adapter, retry/backoff

/packages/service-worker
  Workbox/Servist config, cache rules, update lifecycle

/packages/media
  Media manifests, pending attachments, thumbnail cache, upload progress metadata

/packages/search
  Local lexical search, PGLite-backed search, future hybrid/semantic retrieval

```

Platform capability layer

The app must detect platform/runtime capabilities instead of assuming browser behavior is uniform. iOS Safari, iOS Home Screen PWA, Android Chrome PWA, desktop browser, and future Capacitor shells are different runtimes.

```

type PlatformKind = "ios" | "android" | "desktop";
type RuntimeKind = "browser" | "standalone-pwa" | "capacitor";

type PlatformCapabilities = {
  webPush: boolean;

```

```

badging: boolean;
backgroundSync: boolean;
haptics: boolean;
biometrics: boolean;
shareTarget: boolean;
filesystemAccess: boolean;
opfs: boolean;
webRtc: boolean;
safeAreaInsets: boolean;
};

```

- Never assume Background Sync exists; build foreground/resume/network-restoration retry.
- Treat Web Push as notification delivery, not as reliable background compute.
- Use safe-area primitives on every top/bottom chrome component.
- Expose capability-derived UI states, not broken buttons.

Local-first data flow

```

user action
-> validate locally
-> create signed protocol event
-> write event to Dexie
-> update local materialized view
-> update UI immediately
-> enqueue mutation in outbox
-> sync through bridge/super peer later
-> receive confirmation or competing events
-> verify/apply deterministically

```

Local entity	Store	Purpose
signed_events	Dexie	Raw signed protocol events received or created locally.
mutation_outbox	Dexie	Pending local writes with idempotency keys, retry state, and visible status.
identity_state	Dexie + derived view	Known root/device keys, revocations, delegations, current local identity checkpoint.
contacts_petnames	Dexie	Local petnames, contact cards, trusted introductions, namespace proof cache.
materialized_messages / feeds	Dexie and/or PGLite projection	Fast UI views built from signed events. Rebuildable.
media_manifests / pending_media	Dexie	Manifest refs, thumbnails, upload progress, local pending attachments.
search_projection	PGLite	Local query/search projection over authorized visible data.
sync_offsets	Dexie	Durable Stream offsets, bridge cursors, last applied checkpoints.

Dexie and PGLite division of responsibility

Both Dexie and PGLite are local-first in this design. Dexie is the operational local store for protocol events, outbox, and lightweight views. PGLite is the committed local query/search engine for richer relational projections, full-text search, and eventual hybrid/semantic search. Neither is allowed to replace protocol-level signed events as the source of truth.

Concern	Dexie	PGLite
Signed event persistence	Primary v1 store.	Can mirror selected events if useful for queries.
Mutation outbox	Primary v1 store because it is simple, durable, and easy to inspect.	Not the default.
Simple materialized UI views	Good for conversations, contacts, pending messages, media metadata.	Good once views become relational/query-heavy.
Full-text search	Possible but not preferred long-term.	Preferred committed module.
Hybrid/semantic search	Not preferred.	Preferred foundation, especially with future vector/semantic extensions.
Protocol authority	No.	No.

Service worker and offline policy

- Precache the app shell and critical route chunks.
- Runtime cache icons, static assets, and safe public assets.
- Cache public thumbnails conservatively; use size limits and expiration.
- Do not cache private decrypted data in the service worker cache.

- Do not blindly cache authenticated API responses without privacy and invalidation rules.
- Show a useful offline screen that still exposes local conversations, local contacts, pending outbox, and cached views.
- Show visible pending/sent/failed states. Native-feeling local-first apps do not silently lose writes.

Mutation safety rules

- Every user write must create a local signed event before any network call is required.
- Every mutation must have an idempotency key.
- Retries must use exponential backoff with jitter.
- Retries must run on app foreground/resume and network restoration.
- Abort/cancel handling must prevent stale response application.
- Server/bridge confirmation may mark a local event delivered, but must not rewrite the user event unless the protocol explicitly defines an accepted replacement.
- Failed writes must remain visible and recoverable unless the user deletes them.
- Conflict/revocation errors must be explicit, not hidden as generic network failures.

Framework7 usage rules

- Use Framework7 for app-shell grammar: navigation stacks, tabs, pages, sheets, modals, panels, list views, swipe-back, and platform themes.
- Wrap Framework7 components in /packages/ui where product-specific behavior or consistent tokens are needed.
- Do not let Framework7 routing own protocol state or persistence state.
- Default to iOS theme where platform policy allows; use Material/Android theme mapping when appropriate.
- Test navigation, swipe-back, sheets, keyboard behavior, safe areas, and scroll restoration on real iOS and Android devices.

Design system rules

- Use system font stack for core UI; do not depend on a web font for primary navigation or body text.
- Centralize nav bar height, tab bar height, safe-area padding, card radius, sheet radius, hairline color, material blur, and motion duration tokens.
- Support dark mode from the start.
- Support reduced motion from the start.
- Use glass/translucency sparingly: nav bars, tab bars, sheets, floating controls, media overlays.
- Do not create shallow Apple cosplay: navigation, gestures, scroll behavior, touch targets, and failure states matter more than decorative blur.

PGlite local query/search plan

- PGLite is part of the first PWA architecture, but it should start focused: local relational projections and local search over authorized local data.
- Initial PGLite schema should mirror derived data, not replace signed event persistence.
- Private search must run locally by default.
- Search results must pass current permission/revocation checks before display.
- Future vector/semantic search should be model-versioned, permission-aware, and rebuildable from signed events and manifests.

PGLite v1 projections:

```
profiles_search
contacts_search
messages_search
posts_search
media_search
namespace_records_search
search_index_metadata
```

Later:

```
embeddings
vector_index_metadata
```

hybrid_search_scores
local_rag_context_cache

Testing gates

Gate	Required checks
Local-first write gate	Airplane-mode message/post/contact write updates UI immediately, persists locally, survives refresh, appears in outbox, retries when online.
Verification gate	Tampered event, invalid signature, revoked device key, stale namespace proof, and wrong media hash are rejected before rendering.
Offline shell gate	App loads from cold start offline after first install; local views remain usable; failed network does not white-screen.
Outbox gate	Idempotency prevents duplicate writes after refresh/retry; backoff with jitter is observable in tests.
PGlite gate	Search/projection database can rebuild from signed local events; stale permissions remove or hide results.
Service worker gate	Private decrypted data is not stored in cache storage; update prompt works; offline fallback works.
Framework7 UX gate	Swipe-back, sheets, tab navigation, keyboard behavior, safe areas, and scrolling are tested on real devices.
Performance gate	Initial shell loads fast, long lists virtualize where needed, local writes do not block main thread, search indexing is workerized when heavy.

Build order for the first PWA

Phase	Work	Exit criteria
1. App shell	Create Vite + Framework7 React + TypeScript app, routing, tabs, baseline pages, safe-area shell.	Runs as browser and installable PWA shell; no server dependency for shell boot.
2. Design tokens	Apple-inspired tokens, iOS/Android theme mapping, dark mode, reduced motion.	All shell components consume tokens; no scattered magic UI constants.
3. Local protocol store	Dexie schema, signed_events, mutation_outbox, identity_state, contacts, media manifests, sync offsets.	Offline local writes persist and survive refresh.
4. Local apply pipeline	Validation, signature verification stubs/adapters, deterministic projection reducers.	UI renders from local views, not raw network responses.
5. PGLite projections/search	Add PGLite package, local search schema, rebuild pipeline from signed events.	Local search works offline over authorized local data.
6. Service worker	Workbox/Serwist/vite-plugin-pwa, app shell precache, offline fallback, update lifecycle.	Cold-start offline after prior install; no private decrypted cache leakage.
7. Bridge sync client	Durable Stream/WebSocket reader, outbox submitter, retry/backoff, idempotency, stale-response guard.	Pending writes confirm/retry safely through bridge.
8. Identity/naming UX	Contact cards, petnames, namespace proof UI, trust-source labels.	Users see human names with source-aware trust states.
9. Media UX	Media manifest creation, thumbnail cache, pending upload states, local preview, upload progress.	Media attachments behave locally first and survive offline/refresh.
10. Capacitor readiness	Keep native app boundaries compiling or at least structurally clean.	No architectural blocker to future native packaging.

What not to build yet

- Do not make PWA storage the permanent protocol definition.
- Do not build a full native/Bare/Pear app before the PWA validates UX and protocol boundaries, unless a separate prototype is needed.
- Do not make RxDB the default unless the local document model clearly requires it.
- Do not move private semantic indexing to super peers by default.
- Do not assume iOS background sync exists.
- Do not over-customize Framework7 navigation before testing built-in behavior.
- Do not cache private decrypted payloads in service worker cache storage.
- Do not turn the bridge into the source of truth.

References

Reference	URL
Framework7	https://framework7.io/
Vite	https://vite.dev/
Capacitor docs	https://capacitorjs.com/docs/
Dexie docs	https://dexie.org/docs/
PGLite docs	https://pglite.dev/

Reference	URL
TanStack Query	https://tanstack.com/query/latest
Workbox	https://developer.chrome.com/docs/workbox
Serwist Vite docs	https://serwist.pages.dev/docs/vite/getting-started
MDN Background Synchronization API	https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API
WebKit Web Push for iOS/iPadOS Home Screen apps	https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/
Apple Human Interface Guidelines	https://developer.apple.com/design/human-interface-guidelines
Motion gestures	https://motion.dev/docs/react-gestures
@use-gesture docs	https://use-gesture.netlify.app/docs/gestures/